A Drop at Forty-Nine in Erdős Problem 85

Claude Fable, GPT Sol, Astra, Claude Opus and Robb Walters
Lean Genius project, 2AM Logic

11 October 2026

Abstract

For $n \geq 4$ let $f(n)$ be the least d such that every simple graph on n vertices with minimum degree at least d contains a four-cycle. Erdős Problem 85 asks whether f is eventually nondecreasing. We report two results.

First, $f(48) = 8$ and $f(49) = 7$, a strict drop between adjacent orders, as a computational result. The lower sides are explicit graphs checked in Lean 4 through `native_decide`. For the upper side, Lean refutes one case outright, splits the rest into four strata, excludes two of them, H3 and H5, and reduces H1 and H7 to the unsatisfiability of formulas defined in Lean: 13,351 for H1, and 28 cover and 377,776 leaf formulas for the part of H7 with no triple. The CakeML-verified checker `cake_lpr` accepted an unsatisfiability proof for every one, directly or, for the two hardest, piece by piece through a split composed in Lean: about 28 TB of LRAT for H1 and 20 TB for H7. Lean thus derives the upper side from two explicit unsatisfiability hypotheses, which the external checks support but which are not admitted into Lean; the derivation prints 598 `native_decide` axioms beyond the three standard ones, and each checked file is tied to its Lean formula by emitters, not a kernel proof. To our knowledge it is the first strict drop of f established on the problem's domain at this level of evidence; one drop is compatible with either answer to the problem.

Second, Lean verifies from the standard axioms alone that one proposition, A-REG (for no $k \geq 3$ is there a C_4-free 2^k-regular graph on 4^k vertices), implies a negative answer to Erdős Problem 85. A-REG is a hypothesis with a stated rival; its $q = 4$ analogue is false, as Lean exhibits a C_4-free 4-regular graph on 16 vertices.

1 Introduction

For $n \geq 4$ let $f(n) = \min\{d :$ every simple graph on n vertices with minimum degree at least d contains a $C_4\}$. Erdős Problem 85 (Bloom, n.d.) asks whether $f(n+1) \geq f(n)$ for all sufficiently large n. In Lean 4 (de Moura and Ullrich, 2021), on Mathlib's simple graphs (The mathlib Community, 2020), the function is `minDegreeForC4` and the question is the proposition `Erdos85Question`; Lean proves that its negation is equivalent to the existence of arbitrarily late strict drops (`erdos85Negation_iff_not_question`). A single drop therefore decides nothing about the problem, but it is the first thing a negative answer needs, and it calibrates any theory of where drops occur.

The problem has a Ramsey form. With Boza's convention $r(s) = R(C_4, K_{1,s})$ (Boza, 2024; Zhang et al., 2017), $r(s) \leq N$ holds exactly when $f(N) \leq N - s$, so a plateau $r(s) = r(s+1) = N$ is the same as a drop $f(N) < f(N-1)$. Boza's table gives $r(41) = 49$ and leaves $r(42) \in \{49, 50\}$ open; our upper side selects $r(42) = 49$, which is exactly $f(49) = 7 < 8 = f(48)$. We use f throughout.

Result A. *$f(48) = 8$ and $f(49) = 7$; in particular $f(49) < f(48)$. The lower sides are Lean-checked. The upper side, $f(49) \leq 7$, is derived in Lean by `not_c4FreeMinDegreeWitness_fortyNine_seven_of_h1_h7Evidence` from exactly two hypotheses: h_{H1}, that the 13,351 H1 formulas are unsatisfiable, and h_{H7}, that*

the 28 cover formulas and 377,776 leaf formulas of H7 with $t = 0$ are unsatisfiable; `cake_lpr` *has checked a refutation of every one of these formulas, directly or, for one H1 formula and one H7 leaf, through every piece of a split whose composition is a Lean lemma (Table 2).*

Theorem B (Lean, standard axioms)**.** *Let* A-REG *(*`BinarySquareRegularExclusion`*) state that for every $k \geq 3$ there is no simple C_4-free 2^k-regular graph on 4^k vertices. Then* A-REG $\Rightarrow \neg$ *Erdos85Question (*`not_erdos85Question_of_binarySquareRegularExclusion`*).*

What is new. (1) The drop. To our knowledge no strict drop of f on the problem's domain $n \geq 4$ was previously known: the maintained record lists no partial result and warns that its literature list may be incomplete (Bloom, n.d.), and Boza's table reports no consecutive equality that yields one, though some of its entries are still ranges (Boza, 2024). (2) The check of H1. All 13,351 instances of the largest stratum carry unsatisfiability proofs accepted by the formally verified checker `cake_lpr`, about 28 TB in all, produced and checked without storing the fresh proofs; the one instance that no whole-instance solve finished is closed by a cube partition whose composition is a Lean lemma. Section 3 describes the method, positions it against its closest precedent, the formally verified empty-hexagon theorem (Heule and Scheucher, 2024; Subercaseaux et al., 2024), and explains how a reader can repeat the check. The same method checked the 377,804 formulas to which Lean reduces the part of H7 with no triple (Section 2.3), and Lean excludes the strata H3 and H5 outright, so Lean derives the upper side from the two families of checked formulas alone. (3) Theorem B, which turns the infinite problem into one proposition, together with what is proved beneath it and a map of approaches that fail.

What is not claimed. Result A is not an unconditional theorem of Lean: the `cake_lpr` checks that support h_{H1} and h_{H7} run outside the Lean kernel and are not admitted into Lean, the checked files are tied to the Lean formulas by emitters rather than by a kernel proof, the Lean statement depends on `native_decide`, and a complete cold rebuild of the project sources has not yet been audited. The thirteen packed H7 certificate files (about 2.57 GB) are published with a hash-verified download recipe (Section 8). Table 2 lists, once, every step that separates Result A from a Lean theorem. Nothing is claimed about Erdős Problem 85 itself. A-REG is not asserted; Section 5 gives the evidence on both sides. Every file and Lean module named here is linked to its public location (Section 8), and every number traces to a linked receipt.

2 The drop at forty-nine

2.1 Lower sides

The module `Erdos85FiniteDropWitnesses.lean` contains a C_4-free 7-regular graph on 48 vertices (168 edges) and a C_4-free graph on 49 vertices with minimum degree 6. From them Lean proves $f(48) = 8$ (`minDegreeForC4_fortyEight_eq_eight_checked`) and $f(49) \geq 7$ (`seven_le_minDegreeForC4_fortyNine_checked`), and, taking `hno49` as a hypothesis, $f(48) = 8 \wedge f(49) = 7$ and $f(49) < f(48)$ (`minDegreeForC4_fortyEight_fortyNine_exact_checked`, `minDegreeForC4_fortyNine_lt_fortyEight_checked`); Section 2.2 derives `hno49` from the two unsatisfiability hypotheses of Result A. The graphs are checked through `native_decide`. A cold rebuild from source (pinned Lean 4.31.0 image, fresh build volume) printed, for each witness, three per-declaration axioms of the form `…_native.native_decide.ax_N` beyond `propext`, `Classical.choice` and `Quot.sound`; these express trust in the compiled evaluator. The audit output prints only these per-declaration names; identifying them with `Lean.ofReduceBool` is a statement about how Lean implements `native_decide`, not a line of that output. Theorem B, in the same audit, uses the three standard axioms only. The 48-vertex witness is not isomorphic to any of the ten graphs with 48 vertices and 168 edges in the Afzaly–McKay records (Afzaly and McKay, n.d.)

(verify_boza48_nonisomorphism.py); those records also contain 49-vertex graphs of minimum degree 6, which corroborate $f(49) \geq 7$ but say nothing about the upper side.

2.2 The case split

Let G be C_4-free on 49 vertices with minimum degree at least 7. Two vertices share at most one neighbour, so the non-returning walks of length two from v end at distinct vertices and $\sum_{u \sim v}(\deg u - 1) \leq 48$. Hence every degree is 7 or 8, and a vertex of degree 8 (*high*) has only neighbours of degree 7. The degree sum $7 \cdot 49 + h$ is even, so the number h of high vertices is odd, and Lean proves $h \in \{1, 3, 5, 7, 9\}$ (`orderFortyNine_card_high_eq_one_or_three_or_five_or_seven_or_nine`). The case $h = 9$ is refuted in Lean with no remaining external-evidence hypothesis (`orderFortyNineStratumExcluded_nine`); its proof checks 18 representative certificates through `native_decide`. Then

```
not_c4FreeMinDegreeWitness_fortyNine_seven_of_strata (h₁) (h₃) (h₅) (h₇)
    : ¬ C4FreeMinDegreeWitness 49 7
```

takes exactly the four exclusions of the strata H1, H3, H5 and H7. The case split is therefore part of the formal statement. Lean proves the exclusions of H3 and H5 outright (Section 2.3) and reduces H1 and H7 to unsatisfiability hypotheses, and the module `Erdos85OrderFortyNineCapstoneH3.lean` assembles the result:

```
not_c4FreeMinDegreeWitness_fortyNine_seven_of_h1_h7Evidence (h_H1) (h_H7)
    : ¬ C4FreeMinDegreeWitness 49 7,
```

with corollaries $f(48) = 8 \wedge f(49) = 7$ and $f(49) < f(48)$ (`minDegreeForC4_fortyEight_fortyNine_exact_of_h1_h7Evidence`, `minDegreeForC4_fortyNine_lt_fortyEight_of_h1_h7Evidence`). Its printed axioms are `propext`, `Classical.choice`, `Quot.sound` and 598 `native_decide` axioms, 601 in all, and no `sorryAx`; the two corollaries, which add the witnesses, print 607 and 604 axioms (the same three standard ones and 604 and 601 `native_decide` axioms). They were printed after compiling the capstone from source under Lean 4.31.0 against independently accepted, hash-pinned objects of its 891 imported modules; that is not a cold rebuild of every module. A separate rebuild of the full project closure is in progress: **[PENDING cold-build terminal audit]**.

2.3 The three small strata

Write the high vertices first, so that the adjacency matrix is $\left[\begin{smallmatrix} 0 & B \\ B^{\mathsf{T}} & C \end{smallmatrix}\right]$. High vertices are pairwise non-adjacent, any two of them have exactly one common neighbour, and a low vertex has at most three high neighbours, because its neighbourhood induces a matching. Up to relabelling, the high–low incidence falls into finitely many *cells*, indexed by the number t of low vertices with three high neighbours.

H3. Any two of the three high vertices have exactly one common neighbour, so there are two cells: in $t = 1$ one low vertex is adjacent to all three high vertices, and in $t = 0$ each pair of high vertices has its own common neighbour, these three low vertices spanning a matching with $b \in \{0, 1\}$ edges. Lean proves that every graph of the stratum, after relabelling, satisfies the constraints of one of the two cell representatives (`threeHighCanonicalGraphCover_all` in `Erdos85OrderFortyNineThreeHighOneFiber.lean`), and `orderFortyNineStratumExcluded_three_of_representativeExclusions` reduces the stratum to the exclusion of the two representatives. Lean excludes both representatives by exhaustive finite searches split into 408 parts (24 and 384), each discharged by `native_decide`; the stratum theorem `H3.orderFortyNineStratumExcluded_three` prints the three standard axioms and these 408 `native_decide` axioms. It was assembled from independently audited part objects. A reviewed paper argument (`Q7_H3_PROFILE_EXCLUSION_20260910.md`) excluded H3 before the Lean proof and agrees with it.

H5. There are three cells, $t = 0, 1, 2$. Lean excludes all three: `H5.orderFortyNineStratumExcluded_five` rests on 40 finite search parts and 18 graph-cover and cardinality checks, all discharged by `native_decide`; it prints the three standard axioms and these 58 `native_decide` axioms. Reviewed reductions with a closure ledger (`q7_h5_closure_ledger`) excluded H5 before the Lean proof and agree with it.

H7. The seven high vertices carry a linear triple system with $t \leq 7$ triples (`orderFortyNine_high Incidence_profile_of_seven_high`). A kernel-checked census shows that every such system is equivalent to one of fourteen canonical representatives, and Lean proves that every graph of the stratum satisfies the constraints of one of them (`sevenHighCanonicalGraphCover_all`). The thirteen representatives with $t \geq 1$ are excluded by LRAT certificates checked inside Lean by `native_decide` against the formula `orderFortyNineGeneratedH7SatCnf` (1,329,041 clauses each), and `orderFortyNineStratum Excluded_seven_of_t0` reduces the stratum to the representative with $t = 0$. These thirteen certificate checks appear among the `native_decide` axioms of the capstone. The packed certificate files are available through the download recipe in Section 8. These modules were not replayed in the previous capstone audit, which reused their accepted objects.

In the $t = 0$ cell the 42 low vertices split into 7 with no high neighbour, 14 with one and 21 with two, and the argument works from the graph induced on the seven: it is C_4-free with maximum degree 3 and has 43 isomorphism classes with 6 to 9 edges. Lean excludes 15 of them by a counting argument on common-neighbour pairs, checked by kernel `decide` (`sevenHighT0CountingCertificates_hold`); the other 28 are the *structural cubes*. For each, the cube formula (17,633 variables, 720,825 clauses, the Lean term `orderFortyNineSevenHighT0CanonicalEmptyCubeSatCnf`) is extended by symmetry-breaking clauses for the group $S_7 \ltimes (\mathbb{Z}/2)^7$ acting on the high side; every clause carries a group witness that a small Lean function re-checks, and Lean proves the extension sound. A *cover* formula then splits the extended cube into *leaves*, one per line of a fixed list; at the symmetry-breaking depth 3 used throughout (the theorem takes the depth as a parameter) there are 377,776 leaves over the 28 cubes. Lean proves

$$(\forall \text{ 28 cubes : cover formula UNSAT} \ \wedge \ \forall \text{ leaves : leaf formula UNSAT}) \Longrightarrow \texttt{OrderFortyNineStratumExcluded 7}$$

as `orderFortyNineStratumExcluded_seven_of_structuralHsbEvidence`; the hypothesis, the structure `SevenHighT0CanonicalHsbEvidence` for each cube, is h_{H7} of Result A. The reduction prints the three standard axioms and 91 `native_decide` axioms (`stratum.log`). A reviewed closure ledger (`H7_CLOSURE_20260915.md`) excluded this cell earlier by structural arguments.

The H7 certificates. Each cover and leaf formula was emitted as a DIMACS file, solved by CaDiCaL 3.0.1 (Biere et al., 2020, 2024a) with binary LRAT logging and checked as it streamed by `cake_lpr`, with both binaries pinned by sha256 (`fd601b82…`, `4d47ffdd…`); as for H1 (Section 3.1), leaf proofs were discarded after the check, while the cover and sub-cover formulas and proofs were retained. The base cube files were printed by the Lean term itself, and the symmetry-breaking and cover lines by a native generator whose output matched the Lean definitions byte for byte. One leaf that exceeded the solver time cap was split by lookahead on its edge variables into 256 sub-leaves, each the leaf formula with the units of one cube, and a sub-cover, the leaf formula with the 256 blocking clauses. Lean proves that the leaf is unsatisfiable if the sub-cover and every sub-leaf are (`sevenHighT0CanonicalHsbLeafChecked_of_split`), so the split generator is not trusted. Every formula verified, with no satisfiable formula and no rejected proof. The verified leaf and split checks took 4,269 solver and 706 checker CPU-hours on about 20.2 TB of LRAT (excluding covers and superseded runs; `campaigntotals`). A collector recomputes every formula hash from the pinned inputs and requires a verified receipt for each cover and leaf, or a complete split; a second AI author re-ran it independently with the same result.

2.4 H1 and its 13,351 orbits

H1, one high vertex, carries most of the computation. The eight neighbours of the high vertex induce a perfect matching, every other vertex has exactly one neighbour among them, and the remaining 40 vertices induce a 6-regular C_4-free graph with eight attachment groups of five. Such a graph determines a profile (one of five) and a *miss table* of 24 small counts that record, for pairs of the high vertex's neighbours, how the attachment groups meet. Permutations of the eight neighbours that preserve the matching and the profile act on tables, and Lean proves that every H1 graph can be labelled so that its table is the stored representative of its orbit; so one formula per orbit suffices. An *orbit* below is such a representative together with its SAT formula, the Lean term `oneHighFamilyV2SatCnf` at that table (the hardest has 42,160 variables and 613,228 clauses).

The stored inventory, which Lean reads as a file, has 13,541 orbit representatives; a capacity inequality that Lean proves for every H1 graph removes 190 of them, leaving 13,351 (`oneHighCapacityInventory_total_length`, by `native_decide`). Lean then proves

$$(\forall \text{ orbits } (p, T) \text{ of the } 13{,}351 : \texttt{OneHighFamilyV2CheckedUnsat}\ p\ T) \\ \Longrightarrow \texttt{OrderFortyNineStratumExcluded 1}$$

as `orderFortyNineStratumExcluded_one_of_capacityInventory_checked`, where `OneHighFamilyV2CheckedUnsat` says that the orbit's Lean formula is unsatisfiable. Its cover argument (every H1 graph induces, after relabelling, one of the stored tables) uses `native_decide` for the finite enumeration checks: its printed axioms are `propext`, `Classical.choice` and `Quot.sound` together with 23 `native_decide` axioms from those checks, and no `sorryAx` (`H1_COVER_AXIOMS_20261006.txt`). So, in Lean, H1 reduces to 13,351 SAT questions, and Section 3 answers all of them.

Before any proof was logged, the 1,161 orbits that had no certificate from the first production run in August 2026 were settled verdict-only by a two-solver census (Kissat 4.0.4 (Biere et al., 2024b), then CaDiCaL 3.0.1 (Biere et al., 2020, 2024a); about 5,700 core-hours, no satisfiable instance, no disagreement; `CENSUS.md`). That census is superseded by certificates and we do not rely on it.

3 How H1 was checked

A solver's UNSAT verdict is not a proof, but it can be made into one. DRAT proofs are checked by drat-trim (Wetzler et al., 2014); the LRAT format adds hints that let a small checker replay each step (Cruz-Filipe et al., 2017); and Tan et al. (2021) give `cake_lpr`, an LRAT checker whose correctness, including file parsing, is proved in HOL4 and carried to machine code by the verified CakeML compiler. Large SAT-based results in combinatorics likewise rely on independently checked proofs (Heule, 2018; Heule et al., 2016).

The closest precedent for what we do is the empty-hexagon theorem. Heule and Scheucher (2024) showed by cube-and-conquer (Heule et al., 2012) that every set of 30 points in general position in the plane contains an empty convex hexagon, with the proof checked by `cake_lpr`, and Subercaseaux et al. (2024) verified in Lean that the SAT encoding is faithful to the geometric statement and joined that verification to the `cake_lpr` check. The hexagon verification and ours both reduce the mathematical statement, in Lean, to the unsatisfiability of a formula defined in Lean, and both rely on `cake_lpr` for the unsatisfiability check. For H7, Lean verifies the soundness of the symmetry-breaking extension and the composition of cover and leaf UNSAT hypotheses. This parallels the verified symmetry breaking in the empty-hexagon development (Subercaseaux et al., 2024), while our checked files remain connected to the Lean formulas through emitters. On the encoding-faithfulness axis we claim no advantage: our reduction uses `native_decide` axioms (23 for H1, 598 for the whole upper side; Section 2.2), and we tie each checked CNF file to its Lean formula only through emitters (Section 3.4). What this paper adds is of a different kind: a Lean reduction of a whole stratum to 13,351 formulas, all checked by `cake_lpr` (about 28 TB of proofs), and of a second to 377,804

formulas (about 20 TB); a check-then-discard discipline that keeps only hashes and relies on byte-identical regeneration; the re-validation of an archived certificate bank of 22.55 TB; and a public checker with which a reader can re-check any H1 orbit.

3.1 Check-then-discard

For each orbit the pipeline does three things.

1. *Pin the formula.* Regenerate the CNF from the orbit's table with `v2cnf`, the compiled form of the Lean emitter `Erdos85OneHighV2CnfEmit.lean`, which prints the Lean term `oneHighFamilyV2SatCnf` as DIMACS; run its `check` mode; and require the CNF's sha256 to equal the hash recorded when the orbit was first solved. The emitter binary and container image are pinned by hash.

2. *Check the proof as it streams.* Pipe an LRAT proof, fresh from the solver or read from an archive, through a hashing relay into `cake_lpr` (commit `a36874a8`). An orbit counts only when `cake_lpr` prints `s VERIFIED UNSAT` and, for a fresh solve, the solver reports `s UNSATISFIABLE`. `cake_lpr` exits with status 0 even when a check fails, so that line is the only success signal.

3. *Discard.* Keep the proof's sha256 and length, not the proof. With the identical solver binary the proofs are reproducible byte for byte: three orbits solved twice on different machine types gave identical proofs of 27 to 28.6 GB. A macOS build of the same solver release gives different, equally valid proofs, so reproduction pins the exact binary.

`cake_lpr` reads the proof with a heap fixed in advance, whatever the proof's length: 4 GB sufficed for 1,075 of 1,160 fresh proofs and none needed more than 16 GB, and checking cost about 6% of the solver's CPU time. This bounded memory is what made whole-instance certificates practical; before the run we had planned to cube every hard orbit to keep proofs small.

3.2 Four evidence sets

The 13,351 orbits fall into four disjoint sets (Table 1); their union is the capacity inventory, with no orbit missing.

Table 1: The H1 check. Every orbit has an unsatisfiability proof accepted by the CakeML-verified checker `cake_lpr`; no proof was rejected and no orbit was satisfiable. All checks ran outside the Lean kernel.

Set	Orbits	Proofs	Checker
August 2026 bank	12,094	archived Kissat proofs, converted to LRAT by drat-trim; 22.55 TB, largest 26.5 GB	`cake_lpr`, 8 GB heap
Fresh census	1,160	CaDiCaL 3.0.1 binary LRAT, streamed; 5.64 TB, largest 36.2 GB	`cake_lpr`, 4–16 GB heap
Historical	96	archived DRAT, converted to LRAT by drat-trim; 0.10 TB, largest 2.49 GB	`cake_lpr`
Hardest orbit	1	36 cube proofs, 47.1 GB in all	`cake_lpr`; composed by a Lean lemma
Total	13,351	about 28 TB	

The bank. For 12,094 orbits, the first production run in August 2026 archived a compressed LRAT proof, obtained from a Kissat DRAT proof by drat-trim and then compacted. These proofs had been checked only by drat-trim when produced. We re-checked all of them: regenerate the CNF as above and require its hash to match the producer's ledger; stream the archived file through sha256, decompression, a second sha256

and cake_lpr; and accept only if cake_lpr verifies and both stream hashes match the ledger. All 12,094 verified, with 22.55 TB of LRAT (6.29 TB compressed), the largest proof 26.5 GB, every check within an 8 GB heap, in 408 hours of summed check time. One orbit has an archived proof but no producer ledger line; it is verified from the pinned-emitter CNF and cake_lpr alone, since the ledger match is provenance and not part of the correctness argument. Twenty-one runs reported a CNF mismatch; all were emitter failures on one cloud machine while it was being reclaimed, and every affected orbit re-ran to a verified receipt.

The fresh census. Of the 1,161 orbits with no August certificate, the 1,160 other than the hardest were re-solved with CaDiCaL 3.0.1 (flags `--lrat=true --binary=true`) and checked as they streamed by cake_lpr: 1,157 on AWS Graviton machines with the published Linux arm64 solver binary (sha256 fd601b82…) and a cake_lpr build with sha256 95b64883…, and 3 on a local Apple-silicon machine. All 1,160 verified: 5.64 TB of proofs, 3,435 solver and 207 checker CPU-hours. Of 1,196 run records, 1,163 are verified receipts (three orbits were run twice) and 33 are infrastructure faults (a race in a process watchdog, a 24-hour solver time cap on the longest orbit, and a duplicated claim), each re-run to a verified receipt.

The historical certificates. The 96 orbits that the August run had also certified were re-checked from their archived DRAT proofs: drat-trim converted each to LRAT against the CNF with the expected hash, and cake_lpr (a macOS build of the same commit and assembly file, sha256 d23c413b…) accepted all 96, 0.10 TB of LRAT, the largest proof 2.49 GB. Every converted file has the same sha256 as in an earlier conversion pass. The census and the bank re-check cost about $225 of cloud compute (controller estimates; censusreceipts and bankreceipts).

3.3 The hardest orbit

One orbit, h1_81494a6ef36d3ec9, defeated every whole-instance solve, at caps of up to 24 hours. An adaptive cube-and-conquer procedure split it, by unit-propagation lookahead, into a tree of 36 leaf cubes of depth at most 8. Each split replaces a cube by two that partition it, so the leaves partition the formula.

For the certificate, the 36 leaf formulas were re-solved with binary LRAT logging by the macOS build of CaDiCaL 3.0.1, each leaf's CNF hash-checked against the tree. The proofs were written to files and checked afterwards by that macOS cake_lpr build, with a 4 GB heap; all 36 verified (47.1 GB in all, the largest 8.24 GB). The leaf receipts record each proof's length, but a proof sha256 only for the 30 smaller leaves (up to 2.42 GB).

The Lean module Erdos85H1CubePilot81494a.lean states the tree and the 36 cubes as Lean terms, proves by kernel decide that the cubes cover every cube of the tree (h1Cube81494a_cover), and proves cnf_unsat_of_h1Cube81494a: the orbit's Lean formula is unsatisfiable if all 36 cube formulas are. The generic step cnf_unsat_of_cubeTree and its H1 form oneHighFamilyV2CheckedUnsat_of_cubeTree are in Erdos85CubeTreeComposition.lean. All four statements depend only on propext and Quot.sound (COMPOSITION_AXIOMS_20261006.txt). The leaf hypotheses are supplied by the external checks, so for this orbit the composition is checked in Lean and the leaves outside it.

3.4 What the check establishes

For each of the 13,351 orbits, cake_lpr has accepted a refutation of a CNF file (for the hardest orbit, of 36 cube files) whose hash matches the output of the compiled Lean emitter on that orbit's table. The solvers are no longer trusted. The checks run outside Lean: cake_lpr's guarantee, which covers parsing the files, is a HOL4 theorem about its compiled binary, not a Lean kernel check, and the checks are not admitted into Lean as OneHighFamilyV2CheckedUnsat facts. What remains trusted is cake_lpr's HOL4 specification and compiled binary, and the identity between each checked file and the Lean term oneHighFamilyV2SatCnf, which rests on the compiled emitter and its printing rather than on a Lean proof. A certificate shows that

the checked formula is unsatisfiable, and nothing more. Whether the formula encodes the stratum correctly is a separate question; in H1 the Lean reduction of Section 2.4 answers it for the Lean term, which is why the emitter is the Lean function itself. A reader can re-check any H1 orbit on their own cloud machine with a public machine image, the published bank proofs and a checker kit (Section 8). The H7 checks have the same status and rely on the same `cake_lpr` commit (a different binary build, sha256 `4d47ffdd…`), with one difference in the emitter: the base cube files were printed by the Lean term, while the symmetry-breaking and cover lines come from a native generator whose output was compared byte for byte with the Lean definitions; no public checker image exists for H7.

4 Status

Table 2 is the one place where the paper lists what is checked and what remains open in Lean; the rest of the paper refers to it.

The formal gaps are admitting the H1 and H7 checks into Lean and a kernel link between each checked file and its Lean formula. Reproducibility is a separate issue: the thirteen packed H7 ($t \geq 1$) certificate inputs are published, but a cold rebuild of all project modules has not yet been audited and would still list the `native_decide` axioms. For H1 and H7 both of the first two items are needed: admitting the checks alone would leave open whether the checked files are the Lean formulas. Replaying the LRAT proofs inside Lean would remove both, at a cost we have not attempted.

5 Theorem B and the residue beneath A-REG

The reduction. A pair of adjacent orders gives a drop when it has an existence half and a nonexistence half: a q-regular C_4-free graph on $q^2 - 1$ vertices, together with the nonexistence of a C_4-free graph of minimum degree q on q^2 vertices, gives $f(q^2) < f(q^2 - 1)$ (`PlaneOrderDropWitness.strict_drop`), and cofinally many such pairs refute the problem (`not_erdos85Question_of_cofinalPlaneOrderDrop Family`). For $q = 2^k$ with $k \geq 3$ the existence half is uniform: deleting the absolute nucleus from the even-characteristic polarity graph gives the witness (`Polarity.c4FreeMinDegreeWitness_even_ delete_absolute_nucleus`). On the nonexistence half the normalized square-order reduction is an equivalence (`squareOrderTightCoreExists_iff_witness`), parity forces every tight core to be regular (`squareOrder_regular_of_even`), and Lean proves that A-REG implies the tight-core exclusion (`binarySquareOrderTightCoreExclusion_of_regularExclusion`) and hence $\neg$ Erdos85Question. Every arrow is a dependency of Theorem B and was compiled in the cold audit. Theorem B is a complete checked reduction and not a proof of A-REG; A-REG does not hide further assumptions.

The residue. For a hypothetical q-regular C_4-free graph on q^2 vertices with adjacency matrix A, the *defect* D is defined by $A^2 = (q-1)I + J - D$, and A commutes with D. The components of the defect graph have orders qm_c with $\sum_c m_c = q$, and every vertex of component c has exactly m_c neighbours inside it. For even q there are no components with $m_c = 1$, and for $4 \mid q$ no bipartite components. The Lean statements are `adjMatrix_sq_eq_sub_secondOrderDefect_of_regular`, `binarySquare_regular_exists_defectComponent_partition`, `binarySquare_regular_no_size Q_defectComponent_of_even` and `binarySquare_twoPow_regular_defectGraph_not_bipartite`. What remains is A-REG-NONBIP: partitions $q = \sum_c m_c$ with every $m_c \geq 2$ and every defect component non-bipartite. Its one-component case has a sharp form.

NONBIP-CONNECTED. For binary $q \geq 8$, every loopless q-regular C_4-free adjacency matrix of order q^2 is singular. Equivalently, its defect graph is disconnected, since $\dim\ker A = \#\text{components}(D) - 1$ (`binarySquare_regular_finrank_adj_kernel_eq_card_components_sub_one`).

Table 2: Status of each component of Result A and of Theorem B. "Lean" means a statement elaborated by Lean 4.31.0 from source; where axioms are listed, they are those printed by `#print axioms`.

Component	How it is checked	Open in Lean
Lower sides	Lean, explicit graphs via `native_decide`; cold audit lists three `native_decide` axioms per witness	nothing; not standard-axiom-only
Case split, $h = 9$	Lean (`not_c4FreeMinDegreeWitness_fortyNine_seven_of_strata`); $h = 9$ checks 18 certificates by `native_decide`	nothing; not standard-axiom-only
H1 reduction	Lean: 13,351 orbit formulas suffice (`orderFortyNineStratumExcluded_one_of_capacityInventory_checked`); printed axioms: the three standard ones and 23 `native_decide` axioms, no `sorryAx`	nothing; not standard-axiom-only
H1 orbits	13,351 refutations accepted by `cake_lpr` outside Lean (the hardest as 36 cube leaves, composed by a standard-axiom Lean lemma)	(i) the external checks are not admitted into Lean; (ii) that each checked file is the Lean formula rests on the compiled emitter `v2cnf`, not on a kernel proof
H3 (2 cells)	Lean: `H3.orderFortyNineStratumExcluded_three`; 408 finite search parts by `native_decide`	nothing; not standard-axiom-only
H5 (3 cells)	Lean: `H5.orderFortyNineStratumExcluded_five`; 40 search parts and 18 cover and cardinality checks by `native_decide`	nothing; not standard-axiom-only
H7, $t \geq 1$ (13 cells)	LRAT certificates checked inside Lean by `native_decide`	nothing; not standard-axiom-only; previous capstone audit reused accepted objects
H7, $t = 0$ reduction	Lean: 28 cover and 377,776 leaf formulas suffice (`orderFortyNineStratumExcluded_seven_of_structuralHsbEvidence`)	nothing; not standard-axiom-only
H7, $t = 0$ formulas	refutations of all 377,804 formulas accepted by `cake_lpr` outside Lean (one leaf through a 256-way split composed by a Lean lemma)	(i) the external checks are not admitted into Lean; (ii) that each checked file is the Lean formula rests on the emitters, not on a kernel proof
Upper side	Lean: `not_c4FreeMinDegreeWitness_fortyNine_seven_of_h1_h7Evidence` from h_{H1} and h_{H7}; 3 standard and 598 `native_decide` axioms, no `sorryAx`	the two hypotheses; compiled against accepted dependency objects, not a cold rebuild
Theorem B	Lean, standard axioms only	A-REG is a hypothesis

The Lean target is `BinarySquareConnectedNonbipartiteExclusion`. There is no uniform Lean reduction of the disconnected cases to the connected one, so the connected case and the mixed non-bipartite partitions are separate open cases. Singularity cannot follow from regularity or the spectrum alone; the frontier is to derive it from the full symmetric C_4-free completion.

Evidence on A-REG. The evidence is mixed. In its favour, the existence half is uniform and the defect calculus removes unit and bipartite components. Against it, Lean proves $f(15) = f(16) = 5$ (`minDegreeForC4_fifteen`, `minDegreeForC4_sixteen`), so there is no drop from 15 to 16, and the explicit graph `sixteenRegular` in `Erdos85Problem.lean` is 4-regular (`sixteenRegular_degree`) and C_4-

free (`sixteenRegular_common_le_one`) on 16 vertices: the $q = 4$ analogue of A-REG is false. Every generic attempt at the remaining non-bipartite case has failed (Section 6). The principal rival reading is that special plane orders organize both constructions and obstructions, with no uniform binary exclusion. We hold A-REG as a hypothesis whose truth the finite data do not decide.

6 What does not close A-REG

Several natural approaches to A-REG-NONBIP fail for identifiable reasons, and the failures say what a proof must use: the simultaneous completion of all rows to one symmetric q-regular C_4-free matrix. The full record is in the campaign's proof outline (`FINAL_PROOF_OUTLINE.md`) and cuts ledger (`CUTS_LEDGER_DRAFT.md`); the arguments are reviewed prose unless a Lean name is given.

- *Determinant and spectrum.* The identity $\det(A)^2 = q^4\tau(D)$, with τ the spanning-tree count, makes a square tree count necessary when D is connected, but controls satisfy it without being singular; real spectral conditions likewise admit controls that are not realizable by integral matrices.

- *Inverse signs.* A proposed separation of the signs of inverse entries (P8) is impossible: exact product-sign identities force both signs. It survived a thousand sampled $q = 4$ models only because those models were singular and never instantiated its hypothesis.

- *Structured families.* The interval circulant defect family is excluded for every binary $q \geq 4$ by a cyclotomic argument (`l6-cyclic-trace/UNIFORM.md`), and specific order-256 defects fail local Hasse invariants; neither reduces arbitrary defect graphs to those families. Explicit connected non-bipartite defect graphs pass the alternating square-root test over $\mathbb{F}_2$ but fail the determinant-square condition (`mod2-root-gate/`).

- *Relaxations.* Packing, Hall and fractional-transversal relaxations omit the global completion, and the strongest faithful partial version already has an exact half-integral survivor at $q = 6$; a five-vertex flag relaxation is feasible for every binary $q \geq 16$ (cuts ledger, row 175).

- *Order 64.* At $q = 8$ some defect partitions are closed in Lean, including the size-two sector (`orderSixtyFour_regular_sizeTwo_signedJoint_false_of_connected`), but $[3,3,2]$, $[4,2,2]$ and $[6,2]$ keep non-bipartite cases, $[4,4]$ and $[5,3]$ are only partly analysed and the connected $[8]$ case is open, so 63 to 64 is not a drop.

- *Plane orders 9, 11, 13.* The order-80 search for a 9-regular C_4-free graph ended UNKNOWN in all 20 solver attempts (`Q9_EXISTENCE_DECISION_20260911.md`). A census of Cayley graphs on 52 groups of order 80, 47 of order 120 and 57 of order 168 found no C_4-free Cayley graph of degree 9, 11 or 13 (`CAYLEY_CENSUS_Q11_Q13_20260913.md`); this excludes only those families. With the values of Zhang et al. (2017) and Boza (2024) it leaves $r(109) \in \{120, 121\}$ and $r(155) \in \{168, 169\}$; our literature check found no exact value for either within its stated scope.

7 Collaboration and methods

Reviews. H3, H5 and H7 ($t = 0$) were first excluded by reviewed arguments carried out inside the collaboration room by the AI authors; those arguments were later superseded by the Lean proofs and certificates above. Every Lean result and certificate collection in this paper was accepted only after another AI author independently reviewed it: for Lean results, the pinned sources, build logs, object hashes and printed axioms of the producing build, rebuilt where stated; for the certificate collections, a rerun of the collector over the receipts. None of this was performed by a human mathematician, and the work has not been refereed.

Contributions. Claude Fable led the certification pipeline and cold-integration verification, the lower-side witness graphs, fleet operations, the September 2026 verdict-only census with its cube-partition tooling, the interpretation and the final scope review. GPT Sol and Astra (OpenAI) developed the structural reductions, the negative map and the independent audits; both worked under the shared "sol" seat of the collaboration room. Claude Opus ran the October 2026 H1 certificate check, including the Lean cube-tree composition lemma. Robb Walters set the project's priorities and scope and directed the lab. The shared room infrastructure, which supplied coordination, claims, reviews and a durable transcript (`https://rjwalters.info/rooms/erdos-85`), is acknowledged; it is not an author.

Methods. The work ran over several months and used the strongest frontier models available at each stage, so the model behind a seat changed during the project, which is why GPT Sol and Astra share one seat. Proposals were cheap; admission to the record was not. A Lean result counted only after it elaborated from source under the pinned toolchain in an independent environment and its `#print axioms` output was read, a rule adopted after a stale-cache incident. A solver result counted only with a pinned input hash and a semantic bridge to a formal statement.

Two lessons. *The encoding is the weak link.* At order 64, a first encoding of one endpoint silently reused the shore table of a neighbouring case (80 candidate owners instead of 88) and returned UNSAT for a strengthened, wrong problem. The mismatch was found by comparing the formal shore modes with the certificate universe before the endpoint was called proved; the corrected 88-owner formulas are checked in `Erdos85MuNegThreeZeroFiveCorrectOwnerCertificate.lean`. The H1 pipeline guards against the same failure by emitting each formula with the Lean function that the Lean reduction quantifies over. *Silence is not success.* `cake_lpr` reports a failed check with exit status 0, the P8 separation survived only on models that never met its hypothesis, and long builds that end without an error have not yet printed their axioms. In each case the remedy was to require a positive artifact: a verification line, an instantiated hypothesis, a printed axiom list.

8 Availability

The source code, receipts and tooling linked here are in the Lean Genius repository (`https://github.com/rjwalters/lean-genius`) at commit `92ff6248033c`, under `research/problems/erdos-85-wip-01/` and `proofs/Proofs/`; labels below are relative to those directories.

- H1 bank re-check: `h1_bank_check_20261006/receipts/README.md`, with one receipt per orbit in `h1_bank_check_receipts.tsv` and totals in `h1_bank_check_summary.json`; tooling in `h1_bank_check_20261006/`.
- Fresh census, historical certificates and cube leaves: `h1_cert_full_20261001/receipts/README.md` (pinned tool hashes and a reproduction recipe), `h1_cert_census_receipts.tsv`, `h1_cert_census_summary.json`, `historical96_cake_lpr_receipts.jsonl`, `historical96_receipts.jsonl` (the earlier conversion pass), `pilot_h1_81494a_leaf_cake_lpr.jsonl`; tooling in `h1_cert_full_20261001/` and `h1_cert_pilot_20261001/`; axiom lists of the H1 cover theorem and of the cube composition in `H1_COVER_AXIOMS_20261006.txt` and `COMPOSITION_AXIOMS_20261006.txt`.
- Verdict census and cube tree: `CENSUS.md`, `tree-check.json`; core-hours in `CENSUS_TIMING_20260928.md`; the H1 inventory and the 12,094 bank orbits in `PHASE_B_H1_H3_INVENTORY_20260910.md`.

- Lean: the H1 reduction in `Erdos85OneHighV2CapacityCover.lean` and `Erdos85OneHighV2CapacityInventory.lean`, the inventory file `Certificates/h1_orbit_inventory.compact`; the cold axiom audit in `AXIOM_AUDIT_COLD_20260927/axioms.out`; the strata and small-order names in `STRATA_AND_SMALL_ORDERS_20260928.md`.
- Earlier reviewed arguments for the small strata, superseded by the Lean proofs and certificates (history): `Q7_H3_PROFILE_EXCLUSION_20260910.md` (H3), `H7_POSITIVE_TRIPLE_CELLS_H5_CLOSURE_FORMULA_COUNTS_20260928.md`, `q7_h5_closure_ledger/REVIEWED_RESULT.md`, `H7_CLOSURE_20260915.md` with `h7-closure-ledger-20260915/`, and `Q7_H5_H7_SQUEEZE_20260910.md`.
- Lean upper side and strata: `Erdos85OrderFortyNineCapstoneH3.lean`, re-exported by `Erdos85FiniteDropCapstone.lean`, with its acceptance record `order49_h3_closed_capstone_20261008/README.md` and the release validation `release_assembly_20261011/README.md`; the H3 stratum record `h3_stratum_integration_20261008/`; the H5 audit `h5_terminal_review_20261008/README.md`; the H7 ($t = 0$) reduction in `Erdos85OrderFortyNineSevenHighT0CanonicalHsbStratumCapstone.lean` and the split lemma in `Erdos85OrderFortyNineSevenHighT0CanonicalHsbLeafSplit.lean`.
- Packed H7 ($t \geq 1$) certificates: `fetch-h7-certificates.sh` downloads the thirteen files (about 2.57 GB) to the relative paths read by the Lean modules and verifies them against `strata-lrat.MANIFEST.sha256`. It requires the AWS CLI and authenticated credentials for Requester Pays access; download and request charges may apply. The `cold-buildrecipe` describes the pinned project closure, fresh-object isolation and audit procedure. No completed cold-build result is claimed here.
- H7 ($t = 0$) certificates: `h7_hsb_campaign_20261008/README.md` (formulas, identity receipts, runbook and pinned tool hashes); the collected receipts, one table per cube and `summary.json`, in `h7_hsb_campaign_20261008/receipts/collected/`; the collector `collect_receipts.py`.
- Literature readings and the r-to-f conversion: `manuscript/FIRST_DROP_LITERATURE_CHECK.md`; the order-48 comparison receipt `sat49/BOZA48_NONISOMORPHISM_RECEIPT_20260928.txt`.

Checking H1 yourself. A reader can re-check any H1 orbit on their own AWS machine without downloading anything locally. The public machine image `ami-05697724475f2e748` (region us-east-1) contains the pinned emitter, solver and `cake_lpr` and the `e85-check` tool; the 12,094 bank proofs and the checker kit (manifests, orbit tables, our receipts and pinned binaries) are in an Amazon S3 bucket with Requester Pays access. `e85-check` streams a bank proof into `cake_lpr`, re-solves a census or historical orbit with the pinned CaDiCaL and checks the fresh proof with `cake_lpr` (for the 1,157 census orbits we solved on Linux the proof's sha256 must also equal ours; for the 96 historical orbits and the 3 census orbits solved on macOS there is no comparable proof hash, so the result is a fresh `cake_lpr` check), or rebuilds and checks the cube leaves of the hardest orbit. The guide `CHECKING.md` gives the steps, the bucket paths and the meaning of each result, and `h1_checker_kit/` holds the tooling.

The fresh H1 proofs and the H7 ($t = 0$) leaf and sub-leaf proofs were checked and discarded (the H7 cover and sub-cover proofs are retained in the artifact storage); they are regenerated by running the pinned CaDiCaL binary on the pinned CNF and compared by sha256. Not published: the rest of the certificate storage and the artifact volume with full solver logs and run directories.

References

Narjess Afzaly and Brendan D. McKay. Extremal graphs and Turán numbers. https://users.cecs.anu.edu.au/~bdm/data/extremal.html, n.d. Combinatorial data page; the 49-vertex C_4-free records are labelled as lower-bound examples. Undated web page, accessed 2026-08-25.

Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020. In *Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 51–53. University of Helsinki, 2020.

Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification (CAV 2024)*, Lecture Notes in Computer Science, pages 133–152. Springer, 2024a. doi: 10.1007/978-3-031-65627-9_7.

Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT competition 2024. In *Proceedings of SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions*, volume B-2024-1 of *Department of Computer Science Report Series B*, pages 8–10. University of Helsinki, 2024b.

Thomas F. Bloom. Erdős Problem #85. https://www.erdosproblems.com/85, n.d. Erdős Problems database; undated web page, accessed 2026-09-28.

Luis Boza. Exact values and bounds for Ramsey numbers of C_4 versus a star graph, 2024. URL https://arxiv.org/abs/2409.12770. Version 2, 12 June 2026.

Luís Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt, Matt Kaufmann, and Peter Schneider-Kamp. Efficient certified RAT verification. In *Automated Deduction – CADE 26*, volume 10395 of *Lecture Notes in Computer Science*, pages 220–236. Springer, 2017. doi: 10.1007/978-3-319-63046-5_14.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

Marijn J. H. Heule. Schur number five. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 6598–6606. AAAI Press, 2018. doi: 10.1609/aaai.v32i1.12209.

Marijn J. H. Heule and Manfred Scheucher. Happy ending: An empty hexagon in every set of 30 points. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2024)*, Lecture Notes in Computer Science, pages 61–80. Springer, 2024. doi: 10.1007/978-3-031-57246-3_5.

Marijn J. H. Heule, Oliver Kullmann, Siert Wieringa, and Armin Biere. Cube and conquer: Guiding CDCL SAT solvers by lookaheads. In *Hardware and Software: Verification and Testing (HVC 2011)*, volume 7261 of *Lecture Notes in Computer Science*, pages 50–65. Springer, 2012. doi: 10.1007/978-3-642-34188-5_8.

Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In *Theory and Applications of Satisfiability Testing – SAT 2016*, volume 9710 of *Lecture Notes in Computer Science*, pages 228–245. Springer, 2016. doi: 10.1007/978-3-319-40970-2_15.

Bernardo Subercaseaux, Wojciech Nawrocki, James Gallicchio, Cayden Codel, Mario Carneiro, and Marijn J. H. Heule. Formal verification of the empty hexagon number. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 35:1–35:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/LIPIcs.ITP.2024.35.

Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. cake_lpr: Verified propagation redundancy checking in CakeML. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)*, volume 12652 of *Lecture Notes in Computer Science*, pages 223–241. Springer, 2021. doi: 10.1007/978-3-030-72013-1_12.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429. Springer, 2014. doi: 10.1007/978-3-319-09284-3_31.

Xuemei Zhang, Yaojun Chen, and T. C. Edwin Cheng. Polarity graphs and Ramsey numbers for C_4 versus stars. *Discrete Mathematics*, 340(4):655–660, 2017. doi: 10.1016/j.disc.2016.12.005.